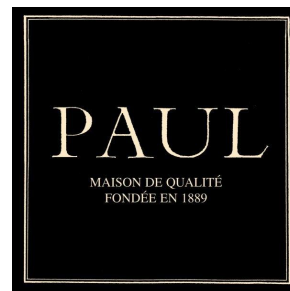




**ÉCOLE MAROCAINE DES
SCIENCES DE L'INGÉNIEUR**
Membre de
HONORIS UNITED UNIVERSITIES



ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR —
CASABLANCA

4^e année en Ingénierie Informatique et Réseaux

Projet de fin d'année

Conception et développement d'un système de prévision des ventes et de planification des approvisionnements pour PAUL Maroc

Réalisé par :

Othmane Moussawi

Encadrant professionnel :

À compléter

Au sein de :

PAUL Maroc

Encadrant pédagogique :

À compléter

Année universitaire : 2025–2026

Dédicaces

À ma famille, pour son soutien constant, sa patience et la confiance qu'elle m'a accordée tout au long de mon parcours. À toutes les personnes qui m'ont encouragé à transformer une problématique métier concrète en une solution utile, fiable et durable. Ce travail leur est dédié avec gratitude.

Remerciements

Nous remercions sincèrement nos encadrants professionnel et pédagogique pour leur disponibilité, leurs orientations et les retours qui ont guidé la conduite de ce projet. Nous exprimons également notre reconnaissance aux responsables et aux équipes opérationnelles de PAUL Maroc, qui ont partagé leurs besoins, leurs contraintes de production et les données nécessaires à la compréhension du processus existant. Enfin, nous remercions le corps professoral de l'École Marocaine des Sciences de l'Ingénieur pour les connaissances techniques et méthodologiques acquises durant notre formation. Ces contributions ont permis de confronter la solution à un contexte réel et d'en améliorer progressivement la pertinence.

Résumé

Ce rapport présente la conception et la réalisation de PrevisionPaul, un système interne destiné à prévoir les ventes d'un point de vente PAUL et à transformer ces prévisions en décisions opérationnelles de production et d'approvisionnement. La solution exploite 373,875 lignes de ventes journalières couvrant la période du 7 juillet 2021 au 28 juin 2026. Elle associe un moteur mensuel, qui compare dix candidats de prévision produit par produit, à un moteur journalier sensible au jour de la semaine, aux changements de niveau, aux fêtes marocaines, aux matchs, aux événements et aux commandes clients connues.

Les volumes prévus sont ensuite rapprochés des recettes et des sous-recettes afin d'estimer les besoins en matières premières, les coûts matière et une quantité recommandée intégrant un niveau de service de 95 %. Un tableau de bord Dash regroupe la production du jour et du mois, les besoins matières, les événements, les commandes, l'historique, les marges et un assistant déterministe fonctionnant entièrement en local. La validation mensuelle glissante disponible atteint un WAPE de 8,25 % sur les quantités agrégées avec Holt–Winters. La version auditée est couverte par 86 tests automatisés réussis.

Mots-clés : prévision des ventes, séries temporelles, planification de production, MRP, nomenclature, Dash, validation glissante, boulangerie.

Abstract

This report presents the design and implementation of PrevisionPaul, an internal system built to forecast sales for a PAUL retail location and turn those forecasts into production and purchasing decisions. The solution processes 373,875 daily sales records spanning from July 7, 2021 to June 28, 2026. It combines a monthly engine, which benchmarks ten forecasting candidates for each product, with a daily engine that accounts for weekday patterns, level shifts, Moroccan holidays, football matches, one-off events and known customer orders.

Forecasted quantities are exploded through product recipes and semi-finished sub-recipes to estimate raw-material requirements, material costs and recommended production quantities at a 95% service level. A Dash web application brings together daily and monthly production, purchasing needs, events, orders, historical analysis, margins and a fully local deterministic assistant. Available walk-forward monthly validation reaches an 8.25% WAPE on aggregate quantities with Holt–Winters. The audited version passes all 86 automated tests.

Keywords : sales forecasting, time series, production planning, MRP, bill of materials, Dash, walk-forward validation, bakery.

Table des matières

Résumé	III
Abstract	IV
Liste des figures	X
Liste des tableaux	XI
Introduction générale	1
Chapitre 1 : Présentation du cadre du projet	3
1.1 Introduction	4
1.2 Présentation de l'organisme d'accueil	4
1.2.1 Contexte général	4
1.2.2 Périmètre métier	5
1.3 Étude de l'existant	5
1.3.1 Description du processus existant	5
1.3.2 Critique de l'existant	6
1.3.3 Contraintes identifiées	6
1.4 Solution proposée	7
1.4.1 Vue d'ensemble	7
1.4.2 Avantages de la solution retenue	7
1.4.3 Originalité du projet	7
1.5 Démarche de développement	8
1.6 Planning prévisionnel	8

1.7 Conclusion	9
Chapitre 2 : Spécification des besoins	10
2.1 Introduction	11
2.2 Identification des acteurs	11
2.3 Spécification des besoins fonctionnels	12
2.3.1 Gestion et préparation des données	12
2.3.2 Prévision mensuelle	13
2.3.3 Prévision journalière	13
2.3.4 Planification des matières et des coûts	14
2.3.5 Pilotage par le tableau de bord	14
2.3.6 Assistant local	14
2.4 Spécification des besoins non fonctionnels	15
2.5 Présentation des cas d'utilisation	16
2.5.1 Diagramme global	16
2.5.2 Cas d'utilisation : consulter la production du jour	17
2.5.3 Cas d'utilisation : relancer la chaîne complète	18
2.5.4 Cas d'utilisation : fiabiliser une recette	18
2.6 Règles métier structurantes	18
2.7 Conclusion	19
Chapitre 3 : Conception du système	20
3.1 Introduction	21
3.2 Modélisation dynamique	21
3.2.1 Séquence de mise à jour quotidienne	21
3.2.2 Activité de génération d'une recommandation	22
3.3 Architecture logicielle	22
3.3.1 Architecture globale	22
3.3.2 Responsabilités des modules	23
3.4 Conception des moteurs de prévision	24

3.4.1	Sélection mensuelle par produit	24
3.4.2	Réconciliation hiérarchique	25
3.4.3	Modèle journalier	26
3.4.4	Combinaison des événements et commandes	26
3.5	Conception du calcul MRP	27
3.5.1	Explosion des nomenclatures	27
3.5.2	Valorisation	27
3.6	Modèle logique des données	27
3.6.1	Relations principales	27
3.6.2	Dictionnaire synthétique	29
3.7	Architecture matérielle et déploiement	29
3.8	Conclusion	30
Chapitre 4 : Réalisation du système		31
4.1	Introduction	32
4.2	Environnement de développement	32
4.2.1	Environnement matériel	32
4.2.2	Environnement logiciel	33
4.3	Réalisation de la chaîne de données	33
4.3.1	Import historique	33
4.3.2	Export SQL défensif	33
4.3.3	Configuration externalisée	34
4.4	Réalisation des prévisions	34
4.4.1	Pipeline mensuel	34
4.4.2	Prévision journalière et suivi	35
4.4.3	Gestion de l'incertitude	35
4.5	Réalisation du calcul matières	35
4.5.1	Recettes et produits semi-finis	35
4.5.2	Exports opérationnels	35
4.6	Principales interfaces graphiques	36

4.6.1	Accueil et synthèse	36
4.6.2	Production journalière	36
4.6.3	Matières premières	37
4.6.4	Événements et commandes	38
4.6.5	Assistant déterministe local	38
4.6.6	Analyse du chiffre d'affaires et des marges	39
4.7	Automatisation et déploiement	40
4.8	Tests et validation	40
4.8.1	Stratégie de test	40
4.8.2	Validation mensuelle	41
4.8.3	Suivi journalier	42
4.9	Difficultés rencontrées et solutions	43
4.9.1	Hétérogénéité et qualité des données	43
4.9.2	Longue traîne et changements de régime	44
4.9.3	Recettes incomplètes et unités	44
4.9.4	Lisibilité de l'interface	44
4.10	Conclusion	44
Conclusion générale		45
Bibliographie et nétographie		46
Bibliographie		46
Annexe A : Guide d'installation et d'exploitation		47
A.1	Installation locale	48
A.2	Commandes d'exploitation	48
A.3	Procédure de mise à jour quotidienne	49
Annexe B : Structure du projet et fichiers produits		50
B.1	Arborescence fonctionnelle	51
B.2	Exports principaux	51

Annexe C : Compléments de validation	52
C.1 Répartition des modèles sélectionnés	53
C.2 Graphiques statiques générés	53
C.3 Limites à contrôler avant usage opérationnel	54

Table des figures

2.1	Diagramme global des cas d'utilisation	16
3.1	Diagramme de séquence de la mise à jour quotidienne	21
3.2	Diagramme d'activité du calcul d'une recommandation	22
3.3	Architecture fonctionnelle globale de PrevisionPaul	23
3.4	Modèle logique des données de PrevisionPaul	28
3.5	Diagramme de déploiement matériel	30
4.1	Page d'accueil du tableau de bord PrevisionPaul	36
4.2	Vue de production journalière	37
4.3	Vue des besoins en matières premières	37
4.4	Gestion des événements et des matchs	38
4.5	Assistant local fondé sur les données calculées	39
4.6	Analyse du chiffre d'affaires et des marges matière	40
4.7	Graphique de validation mensuelle généré par le pipeline	42
C.1	Dashboard statique destiné au responsable	53
C.2	Dashboard statique des matières premières	54

Liste des tableaux

1.1	Périmètre de données observé dans la version auditée	5
1.2	Planning prévisionnel sur douze semaines	8
2.1	Acteurs du système et responsabilités	12
2.2	Besoins non fonctionnels	15
2.3	Description du cas d'utilisation « Consulter la production du jour »	17
2.4	Description du cas d'utilisation « Relancer le calcul complet »	18
3.1	Principaux composants logiciels	23
3.2	Candidats de prévision mensuelle	25
3.3	Dictionnaire des champs principaux	29
4.1	Configuration matérielle de développement et de validation	32
4.2	Environnement logiciel utilisé lors de la validation	33
4.3	Résultats de la validation mensuelle agrégée	41
4.4	Indicateurs observés dans les exports du 7 juillet 2026	43
A.1	Principales commandes d'exploitation	48
B.1	Fichiers produits par le système	51
C.1	Répartition des modèles retenus dans le benchmark	53

Introduction générale

Dans une activité de boulangerie-restauration, la décision de produire ne peut pas être reportée au moment où la demande se manifeste. Les équipes doivent préparer plusieurs références avant l'ouverture ou plusieurs heures avant leur vente, alors que la demande varie selon le jour de la semaine, la saison, les fêtes, les événements et les commandes exceptionnelles. Une quantité trop faible provoque des ruptures et une perte de chiffre d'affaires ; une quantité trop élevée augmente les invendus, immobilise des matières premières et dégrade la rentabilité. Cette tension rend la prévision directement opérationnelle : elle doit être compréhensible, actualisable et reliée aux recettes de production.

Le contexte étudié est celui d'un point de vente PAUL au Maroc. Les informations historiques existent sous plusieurs formes : exports mensuels, états journaliers issus du logiciel de caisse, recettes techniques, prix de matières et événements connus. Avant le projet, ces éléments ne formaient pas une chaîne décisionnelle unique. L'historique permettait d'observer les ventes passées, mais il ne répondait pas automatiquement aux questions quotidiennes : combien produire demain, quelles références sont incertaines, quelles matières commander, quel effet attendre d'une fête ou d'une commande client, et comment contrôler la qualité des prévisions ?

La problématique peut donc être formulée ainsi : *comment concevoir un système local qui transforme des données de vente hétérogènes en prévisions journalières et mensuelles fiables, puis en recommandations de production et d'approvisionnement traçables, tout en restant utilisable par une équipe non spécialiste de la science des données ?*

Pour répondre à cette problématique, nous avons développé PrevisionPaul. La solution réunit une chaîne de préparation de données, plusieurs méthodes de prévision, une validation temporelle sans fuite d'information, des ajustements métier, un calcul MRP à partir des nomenclatures, des exports opérationnels et un tableau de bord web. Le système est conçu pour fonctionner localement afin de préserver la confidentialité des ventes. Il peut également se mettre à jour automatiquement depuis la base SQL de la caisse lorsque l'infrastructure de production est disponible.

Ce rapport est organisé en quatre chapitres. Le premier présente le cadre du projet, l'existant, ses limites, la solution retenue et la démarche de travail. Le deuxième formalise les acteurs, les besoins fonctionnels et non fonctionnels ainsi que les principaux cas d'utilisation. Le troisième

expose la conception dynamique, l'architecture logicielle, le modèle de données et les choix de modélisation. Le quatrième décrit la réalisation, les interfaces principales, le déploiement, les tests et les résultats mesurés. Une conclusion générale synthétise enfin les apports, les limites actuelles et les perspectives d'évolution.

Chapitre 1

Présentation du cadre du projet

Objectifs du chapitre

Ce chapitre situe le projet dans son environnement professionnel. Nous présentons le domaine d'activité, le processus de décision avant informatisation, les limites observées, la solution retenue, son originalité et l'organisation des travaux.

1.1 Introduction

La valeur d'un système de prévision ne dépend pas uniquement de la qualité d'un algorithme. Elle dépend également de la fraîcheur des données, de la manière dont les résultats sont expliqués, du lien avec les contraintes de production et de la capacité des utilisateurs à corriger une situation exceptionnelle. Nous avons donc commencé par comprendre le fonctionnement du point de vente et les décisions qui devaient être améliorées avant de choisir les technologies.

1.2 Présentation de l'organisme d'accueil

1.2.1 Contexte général

PAUL est une enseigne de boulangerie et de restauration dont l'identité de marque rappelle une maison fondée en 1889. Le contexte étudié concerne un point de vente au Maroc. Son activité combine des produits de boulangerie, de viennoiserie, de pâtisserie, de cuisine et de boissons. Cette diversité crée des profils de demande très différents : certains produits sont vendus chaque jour en volume élevé, d'autres sont intermittents, saisonniers ou associés à un moment de consommation précis.

Le projet se concentre sur l'interface entre trois fonctions opérationnelles :

- la **vente**, qui fournit les tickets et les quantités réellement écoulées ;
- la **production**, qui doit préparer les bonnes quantités au bon moment ;
- l'**approvisionnement**, qui doit convertir le plan de production en besoins de farine, beurre, lait, fruits, emballages et autres composants.

Les équipes manipulent également des informations transversales : fêtes marocaines, matchs de l'équipe nationale, événements ponctuels, commandes de clients professionnels, recettes validées par le chef et prix d'achat. La solution devait rassembler ces éléments sans déplacer les données sensibles vers un service externe.

1.2.2 Périmètre métier

Le corpus local audité contient 373 875 lignes de ventes journalières, comprises entre le 7 juillet 2021 et le 28 juin 2026. Il couvre 1 131 libellés produits répartis dans dix familles et représente 4 364 539 unités vendues. Ces ordres de grandeur imposent une automatisation : un traitement manuel produit par produit serait trop lent et difficilement reproductible.

TABLE 1.1 – Périmètre de données observé dans la version auditée

Indicateur	Valeur observée
Période des ventes	07/07/2021 au 28/06/2026
Lignes de ventes journalières	373 875
Jours distincts	1 817
Produits distincts	1 131
Familles de produits	10
Quantité totale enregistrée	4 364 539 unités
Chiffre d'affaires TTC cumulé	57,36 millions MAD

1.3 Étude de l'existant

1.3.1 Description du processus existant

Les ventes historiques proviennent d'abord d'états générés par le logiciel de caisse PI Électronique/Elyx. Une partie de l'historique est disponible sous forme d'exports DOS à largeur fixe et de classeurs mensuels. Les ventes les plus récentes peuvent être extraites depuis SQL Server. Par ailleurs, les recettes techniques et les prix matières sont conservés dans des fichiers distincts. Les décisions de production reposent donc sur plusieurs supports, dont les structures, les noms de produits et les granularités ne sont pas uniformes.

Avant l'intégration proposée, le processus pouvait être résumé comme suit :

1. consulter les ventes passées ou les états de caisse ;
2. estimer les quantités futures à partir de l'expérience récente ;

3. ajuster mentalement cette estimation selon le calendrier ;
4. reporter les quantités dans des supports opérationnels ;
5. rapprocher manuellement les produits de leurs matières premières.

Cette organisation contient une connaissance métier utile, mais elle rend la décision dépendante de la disponibilité d'une personne, ne mesure pas systématiquement l'erreur et complique l'explication d'une recommandation.

1.3.2 Critique de l'existant

L'analyse a mis en évidence les insuffisances suivantes :

- **dispersion des données** : ventes mensuelles, ventes journalières, recettes, événements et prix ne sont pas réunis dans une chaîne unique ;
- **hétérogénéité des sources** : encodage CP850, sous-totaux présents dans certains exports et variations de libellés peuvent introduire des doubles comptes ou des ruptures de séries ;
- **prévision uniforme** : une même méthode ne convient ni à un produit régulier, ni à une vente intermittente, ni à un produit récemment introduit ;
- **absence de validation temporelle systématique** : une précision calculée sur les données d'entraînement surestime la performance réelle ;
- **prise en compte informelle des événements** : les fêtes, matchs et commandes importantes ne sont pas intégrés de manière reproductible ;
- **rupture entre vente et achat** : une prévision de produits finis ne fournit pas directement la quantité de matières à commander ;
- **faible traçabilité** : il est difficile de savoir quel modèle, quelle recette ou quel ajustement a conduit à une valeur donnée.

1.3.3 Contraintes identifiées

Les contraintes métier ont orienté l'architecture. Les données commerciales doivent rester sur la machine de l'entreprise. Le système doit fonctionner sur un poste Windows, supporter une mise à jour par fichiers lorsque la connexion SQL n'est pas disponible, ne pas écraser l'historique si une extraction est vide ou aberrante, et fournir des exports compréhensibles par des utilisateurs non techniques. Enfin, les recettes restent en cours de fiabilisation ; le système doit donc distinguer une recette exacte d'une estimation et rendre cette incertitude visible.

1.4 Solution proposée

1.4.1 Vue d'ensemble

La solution retenue est une application locale en Python composée d'une chaîne de traitement et d'un tableau de bord web. Elle conserve les données métier dans des fichiers CSV et JSON lisibles, tout en proposant un accès automatisé en lecture à la base SQL de la caisse. Elle produit deux niveaux de prévision complémentaires :

- un système **mensuel**, destiné au plan de production, au budget et au calcul MRP ;
- un système **journalier**, destiné à la préparation quotidienne et au suivi court terme.

La chaîne complète va de la vente réelle à la recommandation : préparation des données, sélection du modèle, ajustements calendrier et commandes, estimation de l'incertitude, explosion des nomenclatures, calcul des coûts, génération d'exports et restitution dans le dashboard.

1.4.2 Avantages de la solution retenue

Cette solution présente plusieurs avantages. Elle est reproductible, car un même jeu de données et une même configuration conduisent au même résultat. Elle est traçable, car le modèle retenu par produit, la source des recettes et les ajustements sont externalisés. Elle est évolutive, car les modules de prévision, de nomenclature et d'interface peuvent être améliorés séparément. Elle est enfin compatible avec le niveau d'équipement existant : un poste Windows et un navigateur suffisent pour consulter l'application.

1.4.3 Originalité du projet

L'originalité ne réside pas dans l'emploi isolé d'un modèle statistique. Elle provient de l'enchaînement de plusieurs mécanismes métier dans une même solution :

- comparaison de dix candidats de prévision et choix différent selon le produit ;
- réconciliation entre la somme des produits et la prévision agrégée ;
- apprentissage de l'impact des matchs à partir des jours comparables ;
- neutralisation des pics assimilables à de grosses commandes dans l'apprentissage, puis ajout exact des commandes futures connues ;
- conversion récursive des produits semi-finis en matières de base ;

- assistant guidé local, sans modèle génératif ni appel réseau, dont chaque réponse provient des fichiers calculés ;
- articulation entre précision statistique, marge de sécurité et usage opérationnel.

1.5 Démarche de développement

Nous avons adopté une démarche itérative proche d'un cycle incrémental. Chaque itération produit un résultat testable : un import fiable, une prévision, un export, une nouvelle vue du dashboard ou une amélioration des recettes. Ce choix est plus adapté qu'un cycle strictement séquentiel, car la qualité des données et les retours métier modifient régulièrement les règles de calcul.

Le cycle d'une itération comprend cinq étapes : observer le besoin, isoler les données concernées, implémenter une règle ou un modèle, vérifier par test et backtest, puis présenter le résultat dans un format exploitable. Le dépôt Git conserve les évolutions et les correctifs. La version synchronisée comprend 22 commits depuis l'import initial public du code, avec une attention particulière à l'automatisation SQL et à la fiabilisation des recettes.

1.6 Planning prévisionnel

Le tableau 1.2 présente le découpage prévisionnel adopté. Les semaines sont volontairement exprimées relativement au démarrage afin que le planning reste cohérent avec les dates administratives du stage.

TABLE 1.2 – Planning prévisionnel sur douze semaines

Étape	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12
Analyse du besoin et des sources												
Nettoyage et consolidation des ventes												
Prototype de prévision mensuelle												
Prévision journalière et événements												
Nomenclatures et besoins matières												
Tableau de bord et exports												
Automatisation et déploiement												
Tests, validation et documentation												

1.7 Conclusion

Ce chapitre a montré que le besoin dépasse une simple visualisation de ventes. Le problème consiste à relier des sources hétérogènes à une décision de production et d'achat, en mesurant l'incertitude et en conservant la confidentialité. La solution proposée répond à ce cadre par une architecture locale, modulaire et itérative. Le chapitre suivant traduit ce cadre en besoins précis et en cas d'utilisation.

Chapitre 2

Spécification des besoins

Objectifs du chapitre

Ce chapitre formalise les services attendus de PrevisionPaul. Nous identifions les acteurs, les besoins fonctionnels et non fonctionnels, puis nous décrivons les cas d'utilisation qui structurent l'application.

2.1 Introduction

La spécification sert de contrat entre le problème métier et la réalisation technique. Elle évite de confondre une fonctionnalité utile avec un choix d'implémentation. Dans notre cas, le besoin primaire est d'aider une équipe à décider quoi produire et quoi commander. Les modèles, les fichiers et le framework web sont des moyens au service de cette décision.

2.2 Identification des acteurs

La solution est un outil interne mono-organisation. Elle distingue néanmoins plusieurs rôles selon les actions réalisées.

TABLE 2.1 – Acteurs du système et responsabilités

Acteur	Responsabilité principale	Interactions
Responsable de production	Préparer les quantités quotidiennes et contrôler les références sensibles.	Consulter le plan du jour et de la semaine, filtrer, exporter, appliquer une correction.
Gestionnaire / responsable	Piloter le plan mensuel, les achats, les coûts et les alertes.	Consulter les KPI, le MRP, les marges, les validations et les exports.
Chef / référent recettes	Fiabiliser les nomenclatures et les quantités de matières.	Valider ou corriger les recettes, compléter les prix et contrôler la couverture.
Administrateur technique	Maintenir les sources et l'exécution automatisée.	Importer les ventes, lancer le pipeline, surveiller les journaux et mettre à jour le code.
Système de caisse Elyx	Fournir les ventes réelles clôturées.	Exposer les lignes de tickets via SQL Server ou des exports fichiers.
Planificateur Windows	Exécuter la chaîne sans intervention humaine.	Lancer l'export SQL, la prévision journalière et la prévision mensuelle à 05 h 30.

2.3 Spécification des besoins fonctionnels

2.3.1 Gestion et préparation des données

Le système doit permettre de :

- importer les ventes journalières depuis les exports historiques ou la base SQL de la caisse ;
- normaliser les dates, codes, produits, familles, quantités et chiffres d'affaires ;
- supprimer les lignes de sous-total et éviter les doubles comptes ;
- fusionner les ventes SQL récentes avec l'historique sans perdre les dates antérieures ;
- refuser une mise à jour vide ou manifestement aberrante ;
- signaler la date de fraîcheur des dernières ventes.

2.3.2 Prévision mensuelle

Le moteur mensuel doit :

- agréger l'historique par mois et compléter les mois récents depuis la source journalière ;
- calculer plusieurs prévisions par produit : ARIMA, Croston, ETS, ensembles, moyenne, moyenne mobile, naïf saisonnier, Theta et modèle global de gradient boosting ;
- sélectionner le modèle le plus performant pour chaque produit à partir d'un benchmark hors échantillon ;
- réconcilier la somme des prévisions produits avec une prévision agrégée ;
- appliquer les fêtes, événements, matchs et commandes clients ;
- produire une fourchette basse et une quantité recommandée selon le niveau de service ;
- générer un plan mensuel, des synthèses par famille et des fichiers de validation.

2.3.3 Prévision journalière

Le moteur journalier doit :

- produire un horizon glissant de 62 jours par produit ;
- combiner un niveau récent avec une ancre annuelle ;
- apprendre les poids des jours de semaine ;
- détecter les ruptures de niveau récentes et adapter le poids de l'information la plus fraîche ;
- appliquer les multiplicateurs de fête, de match et d'événement sans les empiler de manière excessive ;
- neutraliser les pics assimilables à une commande exceptionnelle dans la série d'apprentissage ;
- ajouter les commandes futures connues aux dates correspondantes ;
- calculer une fiabilité sur une validation glissante de 28 jours ;
- autoriser une correction manuelle par facteur ou quantité fixe.

2.3.4 Planification des matières et des coûts

À partir des quantités prévues, le système doit :

- associer chaque produit à une recette exacte ou, à défaut, à une recette estimée explicitement signalée ;
- reconnaître les alias de matières et homogénéiser les unités ;
- éclater récursivement les produits semi-finis en matières premières de base ;
- exclure du bon de commande les familles revendues et les ressources non achetées, par exemple l'eau du réseau ;
- calculer les quantités mensuelles par ingrédient et par département ;
- estimer le budget d'achat et le food-cost lorsque les prix sont connus ;
- classer les recettes manquantes selon leur poids dans les achats afin de prioriser le travail du chef.

2.3.5 Pilotage par le tableau de bord

L'interface, construite avec les composants de mise en page Dash [7], doit regrouper cinq espaces : Accueil, Production, Événements et commandes, Assistant, Analyse. Elle doit proposer des indicateurs synthétiques, des filtres, des tableaux paginés, des courbes, des alertes et des exports Excel. Une carte de navigation depuis l'accueil doit conduire directement à la vue demandée. Un bouton unique doit pouvoir recalculer les prévisions journalières et mensuelles après une modification.

2.3.6 Assistant local

L'assistant doit répondre à des questions structurées sur la production d'un jour, la prévision d'un produit, son historique, les matières premières, les événements et la fiabilité. Les réponses doivent être déterministes et provenir des fichiers locaux. Aucun appel à un modèle externe ou à une API réseau ne doit être effectué.

2.4 Spécification des besoins non fonctionnels

TABLE 2.2 – Besoins non fonctionnels

Qualité	Exigence	Critère de vérification
Confidentialité	Les ventes et recettes restent sur le poste ou le serveur de l'entreprise.	Assistant sans dépendance réseau ; aucune donnée métier suivie dans Git.
Fiabilité	Une source SQL vide ou aberrante ne doit pas détruire l'historique.	Contrôles défensifs avant fusion et écriture.
Exactitude	La performance doit être mesurée sur des périodes postérieures aux données d'entraînement.	Backtest annuel et validation walk-forward.
Traçabilité	Les modèles, recettes, prix et ajustements doivent être identifiables.	Fichiers JSON de configuration et exports détaillés.
Utilisabilité	Un utilisateur non technique doit atteindre la bonne vue et comprendre l'indicateur.	Navigation guidée, aides contextuelles et libellés métier.
Maintenabilité	Les responsabilités doivent être séparées par modules.	Modules dédiés aux données, modèles, fêtes, MRP, coûts et reporting.
Portabilité	L'application doit fonctionner sur l'environnement Windows existant.	Lanceurs double-clic, chemins contrôlés, tâche planifiée Windows.
Performance	Le recalcul doit rester compatible avec une utilisation quotidienne.	Cache des lectures coûteuses et calcul nocturne automatisé.
Testabilité	Les règles critiques doivent être vérifiables automatiquement.	Suite Pytest ; 86 tests réussis sur la version audité.
Résilience	Une dépendance optionnelle absente ne doit pas bloquer tout le pipeline.	Replis prévus pour Prophet et Holt-Winters.

2.5 Présentation des cas d'utilisation

2.5.1 Diagramme global

La figure 2.1 représente les interactions majeures selon la notation UML [2]. Les acteurs humains sont séparés des systèmes techniques afin de rendre visible l'automatisation nocturne.

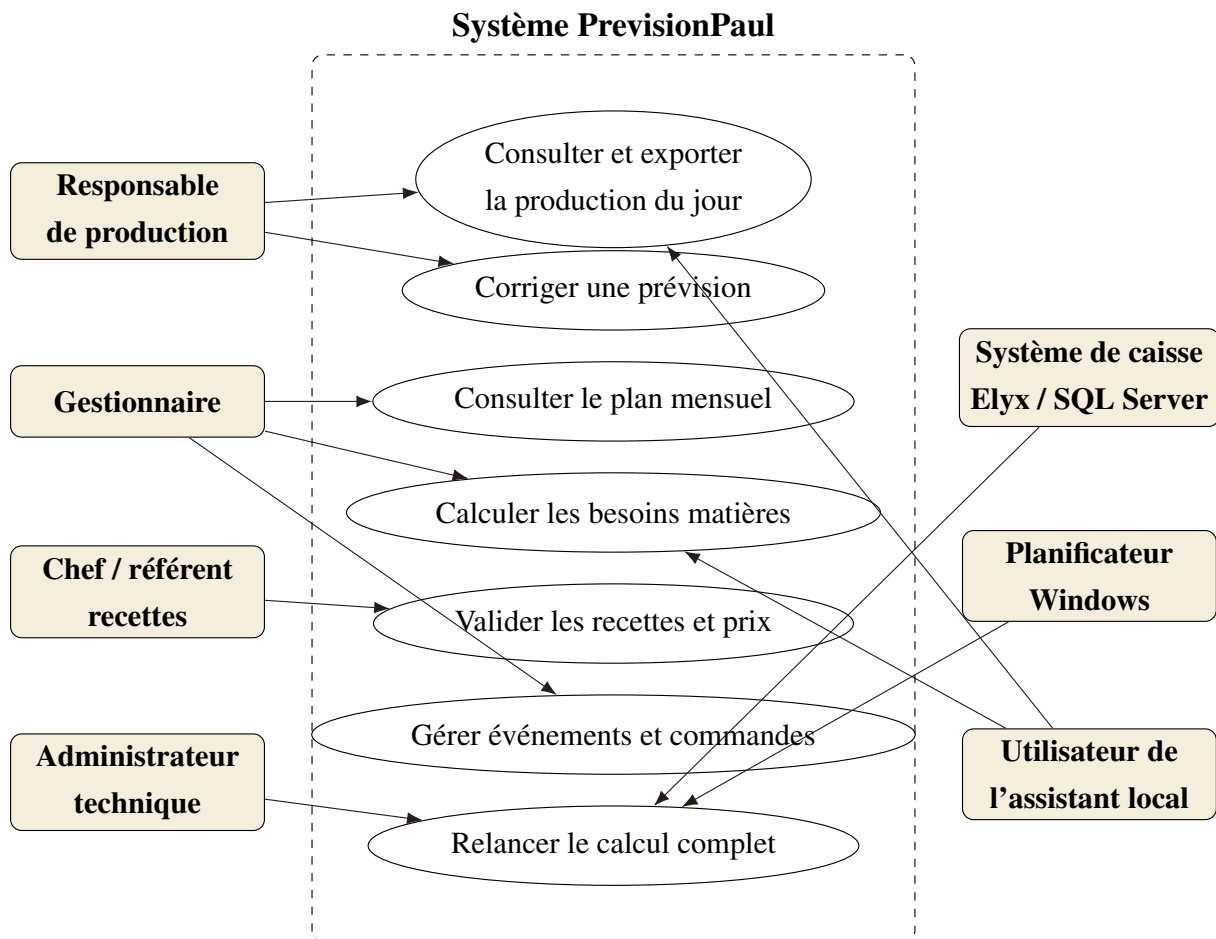


FIGURE 2.1 – Diagramme global des cas d'utilisation

2.5.2 Cas d'utilisation : consulter la production du jour

TABLE 2.3 – Description du cas d'utilisation « Consulter la production du jour »

Rubrique	Description
Acteur principal	Responsable de production.
Objectif	Obtenir les quantités recommandées par produit pour une date donnée.
Préconditions	Les ventes ont été importées et les prévisions journalières ont été générées.
Déclencheur	L'acteur ouvre Production, puis l'onglet Du jour.
Scénario nominal	1) Le système propose la date courante ; 2) l'acteur choisit une date et éventuellement une catégorie ; 3) le système affiche les quantités, la prévision de base et la fiabilité ; 4) l'acteur filtre ou exporte le tableau Excel.
Scénario alternatif	Si les données sont anciennes, une alerte de fraîcheur est affichée. Si une valeur paraît incohérente, l'acteur applique un facteur ou une quantité fixe puis régénère les prévisions.
Postconditions	Le plan est consulté ou exporté ; une correction éventuelle est enregistrée dans la configuration locale.

2.5.3 Cas d'utilisation : relancer la chaîne complète

TABLE 2.4 – Description du cas d'utilisation « Relancer le calcul complet »

Rubrique	Description
Acteurs	Administrateur technique ou planificateur Windows.
Objectif	Mettre à jour les ventes, les prévisions, le plan de production et le MRP.
Préconditions	Python et les dépendances sont installés ; les dossiers de données sont accessibles.
Scénario nominal	1) Extraire les ventes SQL clôturées ; 2) fusionner l'historique ; 3) générer la prévision journalière et le suivi ; 4) exécuter le pipeline mensuel ; 5) écrire les exports datés ; 6) journaliser l'exécution.
Scénario alternatif	Si SQL ne renvoie aucune donnée fiable, conserver le CSV existant et poursuivre avec le dernier historique valide. En cas d'échec d'une étape critique, écrire l'erreur dans le journal et interrompre les étapes dépendantes.
Postconditions	Les fichiers d'export sont actualisés et le dashboard lit le dossier daté le plus récent.

2.5.4 Cas d'utilisation : fiabiliser une recette

Le chef reçoit un classeur organisé par catégorie. Chaque produit présente sa recette exacte lorsqu'elle existe, sinon une estimation et sa provenance. Le chef corrige les composants, valide la ligne, puis l'administrateur importe les recettes confirmées. Le système conserve une sauvegarde horodatée, met à jour la provenance et régénère le MRP. Ce cas d'utilisation est essentiel : une bonne prévision de produits finis ne suffit pas si la nomenclature matière est inexacte.

2.6 Règles métier structurantes

Les règles suivantes sont traitées comme des invariants du système :

1. les ajustements positifs et négatifs liés au calendrier sont combinés en conservant le plus fort de chaque sens, afin d'éviter un empilement irréaliste ;
2. une commande client future connue est ajoutée telle quelle, mais les pics comparables du passé sont neutralisés pendant l'apprentissage ;

3. une quantité recommandée ne peut pas être inférieure à zéro ;
4. l'étiquette de fiabilité dépend d'une erreur hors échantillon ou signale explicitement un historique court ;
5. une recette estimée ne doit jamais être présentée comme exacte ;
6. le dashboard lit le dossier d'export daté le plus récent, tandis que les fichiers journaliers de suivi restent à la racine du dossier exports ;
7. aucune question de l'assistant ne peut déclencher une communication réseau.

2.7 Conclusion

Les besoins définis dans ce chapitre couvrent la chaîne complète, depuis la collecte des ventes jusqu'à l'usage quotidien des recommandations. Ils imposent une séparation claire entre données, prévision, ajustements métier, calcul matière et restitution. Le chapitre suivant présente la conception retenue pour satisfaire ces exigences et les échanges entre ses composants.

Chapitre 3

Conception du système

Objectifs du chapitre

Ce chapitre répond à la question « comment construire la solution ? ». Nous présentons les scénarios dynamiques, l'architecture en couches, les modèles de prévision, la conception du MRP, le modèle logique des données et le déploiement.

3.1 Introduction

La conception de PrevisionPaul repose sur une idée centrale : séparer la donnée source, les calculs métier et la restitution. Cette séparation permet de rejouer un calcul, de tester chaque règle indépendamment et de remplacer un composant sans réécrire toute l'application. Les fichiers CSV et JSON forment un contrat simple entre les modules ; le dashboard n'entraîne pas lui-même les modèles, il lit les résultats produits par les moteurs.

3.2 Modélisation dynamique

3.2.1 Séquence de mise à jour quotidienne

La figure 3.1 détaille le scénario automatisé. L'extraction SQL est volontairement placée avant les deux moteurs. Les fichiers journaliers sont générés avant le pipeline mensuel, car ce dernier peut compléter son panel avec les mois complets issus de la source journalière.

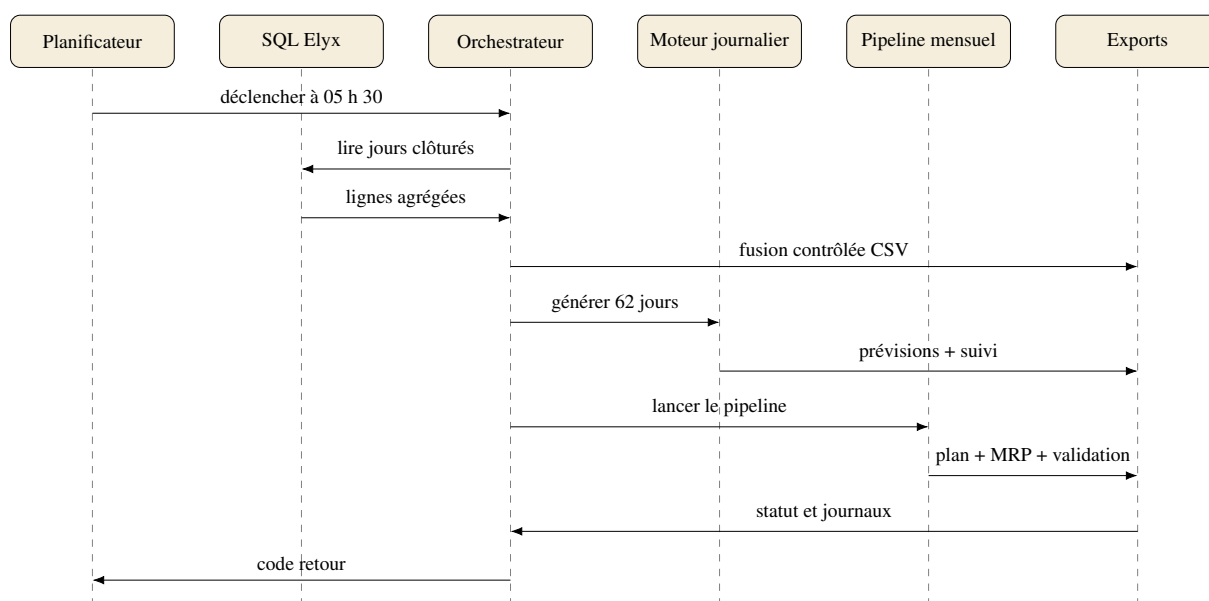


FIGURE 3.1 – Diagramme de séquence de la mise à jour quotidienne

3.2.2 Activité de génération d'une recommandation

Le calcul d'une recommandation suit une série de contrôles. Une vente future ne doit jamais être mélangée aux données d'entraînement et un produit sans historique suffisant doit être signalé au lieu de recevoir une fausse précision.

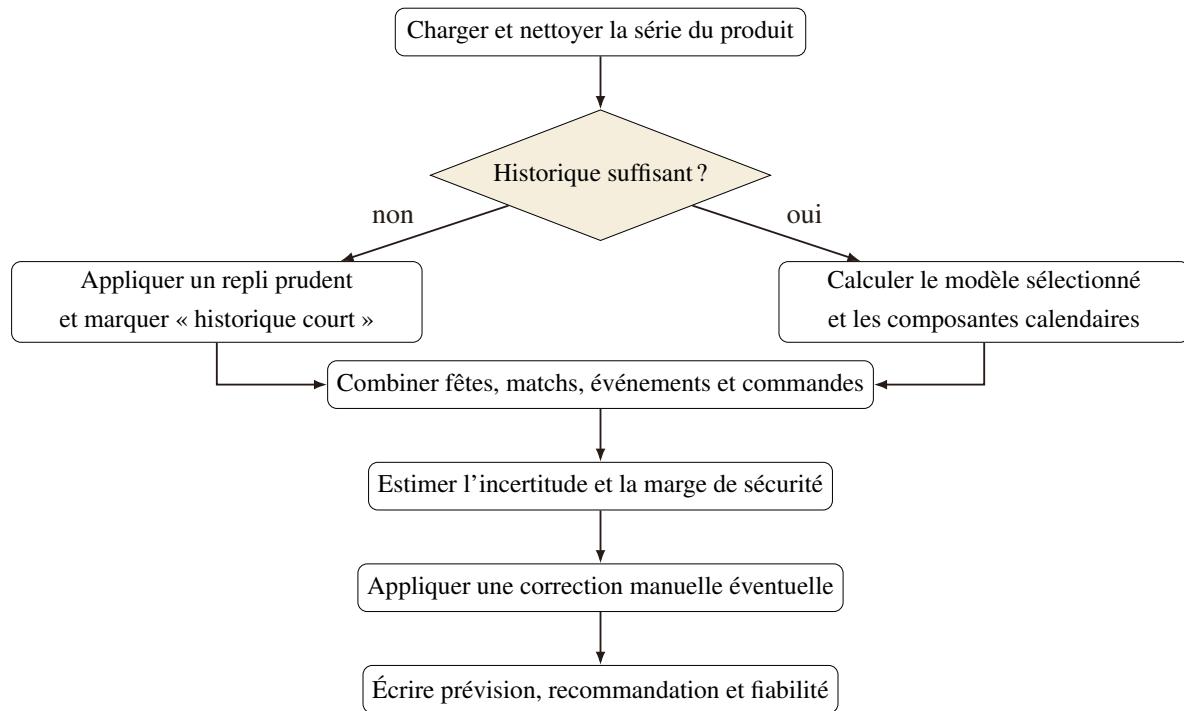


FIGURE 3.2 – Diagramme d'activité du calcul d'une recommandation

3.3 Architecture logicielle

3.3.1 Architecture globale

L'architecture de la figure 3.3 distingue six zones : sources, ingestion, moteurs, règles métier, sorties et interface. Les flèches pleines représentent le flux nominal ; la configuration JSON agit transversalement sur les moteurs et les règles.

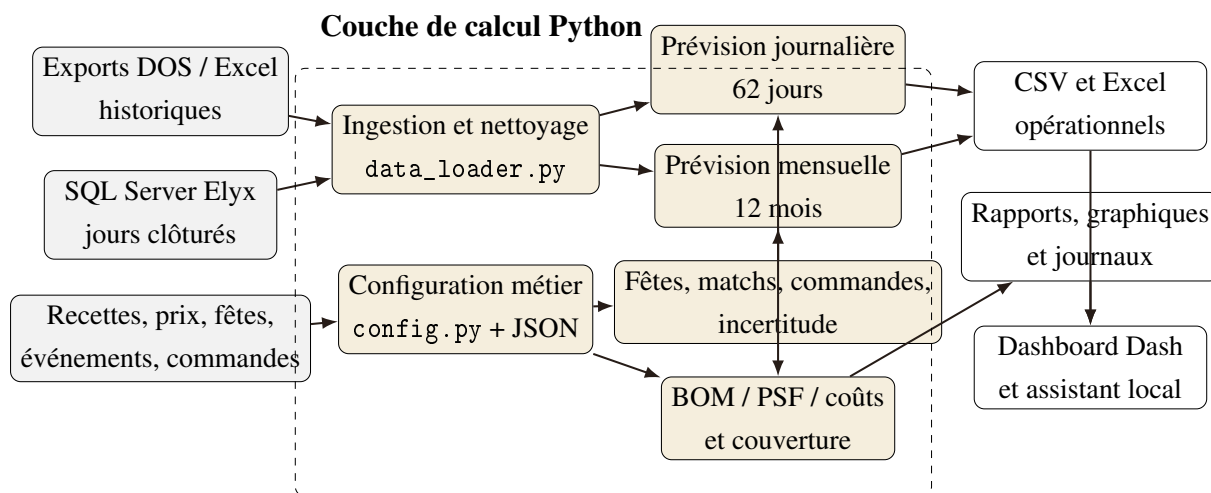


FIGURE 3.3 – Architecture fonctionnelle globale de PrevisionPaul

3.3.2 Responsabilités des modules

Le code principal est réparti dans le paquet `paul_forecast`. Le tableau 3.1 synthétise les responsabilités. Cette organisation joue le rôle d'un diagramme de composants statique : les dépendances vont de la configuration vers les services métier, puis vers le pipeline et l'interface.

TABLE 3.1 – Principaux composants logiciels

Composant	Responsabilité
<code>data_loader.py</code>	Charge, valide, nettoie et agrège les ventes mensuelles et journalières.
<code>forecasting.py</code>	Implémente les prévisions de base et les replis lorsque certaines bibliothèques sont absentes.
<code>modele_global.py</code>	Entraîne un modèle de gradient boosting partagé entre les produits à partir de variables retardées et calendaires.
<code>forecast_journalier.py</code>	Calcule le niveau récent, l'ancre annuelle, le profil hebdomadaire, les ruptures et l'horizon de 62 jours.
<code>saisonnalite_fetes.py</code> / <code>calibration_fetes.py</code>	Mesure et applique les profils des fêtes marocaines.
<code>matchs.py</code> / <code>evenements.py</code>	Apprend ou applique les impacts des matchs et événements ponctuels.
<code>commandes.py</code>	Détecte les pics assimilables à des commandes, nettoie l'apprentissage et ajoute les commandes futures.
<code>incertitude.py</code>	Convertit les erreurs mesurées en intervalles, quantités recommandées et étiquettes de fiabilité.

Composant	Responsabilité
<code>bom.py</code>	Associe les produits aux recettes, éclate les PSF et normalise les matières.
<code>couts.py / marges.py</code>	Valorise les besoins et calcule les indicateurs de marge matière.
<code>couverture.py</code>	Mesure la part des volumes couverte par des recettes exactes ou estimées.
<code>backtest.py / suivi.py</code>	Évalue les modèles sans fuite temporelle aux niveaux mensuel et journalier.
<code>pipeline.py</code>	Orchestre la prévision mensuelle, la réconciliation, le MRP, la validation et les exports.
<code>assistant.py</code>	Fournit des outils de question-réponse déterministes à partir des fichiers locaux.
<code>reporting.py</code>	Génère tableaux, classeurs, bons de commande et graphiques statiques.
<code>dashboard.py</code>	Construit l'interface Dash, ses vues, ses callbacks et ses exports interactifs.

3.4 Conception des moteurs de prévision

3.4.1 Sélection mensuelle par produit

Un seul modèle global appliqué à toutes les références serait inadapté. Nous comparons sept modèles de base et trois candidats complémentaires, soit dix candidats : ETS/Holt–Winters, Theta, ARIMA, naïf saisonnier, moyenne mobile, moyenne historique, Croston, ensemble moyen, ensemble médian et gradient boosting global. Ces familles sont décrites dans la littérature de référence sur la prévision [1] et dans les documentations de statsmodels [5] et scikit-learn [6]. Croston cible notamment les demandes intermittentes ; le modèle naïf saisonnier sert de référence ; les ensembles réduisent la sensibilité à un choix unique ; le gradient boosting mutualise l'information entre produits.

TABLE 3.2 – Candidats de prévision mensuelle

Candidat	Principe	Profil adapté
ETS / Holt–Winters	Niveau, tendance et saisonnalité exponentiels.	Séries régulières avec saisonnalité annuelle.
Theta	Décomposition de la série en lignes Theta.	Séries lisses ou modérément tendanciennes.
ARIMA	Dépendances autorégressives et moyennes mobiles.	Séries avec autocorrélation exploitable.
Naïf saisonnier	Reprend la valeur du même mois de l'année précédente.	Saisonnalité stable et forte.
Moyenne mobile	Moyenne des derniers mois.	Niveau récent stable.
Moyenne	Moyenne historique.	Référence simple ou historique court.
Croston	Sépare taille et intervalle des demandes non nulles.	Produits intermittents.
Ensemble moyen	Moyenne de plusieurs modèles statistiques.	Réduction du risque d'un modèle isolé.
Ensemble médian	Médiane robuste de quatre candidats.	Présence de prévisions extrêmes.
GBM global	Apprentissage partagé sur retards, calendrier, fêtes et famille.	Produits bénéficiant de motifs communs.

Le benchmark utilise douze fenêtres glissantes à un mois d'horizon. Pour chaque produit p et fenêtre t , l'erreur absolue est :

$$e_{p,t,m} = |y_{p,t} - \hat{y}_{p,t,m}|, \quad (3.1)$$

où m désigne le modèle candidat. Le modèle retenu minimise la moyenne des erreurs disponibles. Le fichier `modele_par_produit.json` conserve ce choix pour 848 produits disposant d'un historique suffisant.

3.4.2 Réconciliation hiérarchique

La somme de plus de mille prévisions ajustées séparément peut diverger du niveau global. Nous calculons donc une prévision agrégée Holt–Winters, puis un facteur de réconciliation mensuel :

$$r_t = \text{clip} \left(\frac{\hat{Y}_t^{\text{global}}}{\sum_p \hat{y}_{p,t}}, 0,7, 1,4 \right). \quad (3.2)$$

La prévision réconciliée devient $\hat{y}_{p,t}^{\text{rec}} = r_t \hat{y}_{p,t}$. Les bornes évitent qu'un agrégat aberrant ne transforme excessivement toutes les références.

3.4.3 Modèle journalier

Le moteur journalier cherche un compromis entre réactivité et saisonnalité. Pour un jour futur d , il calcule :

$$\hat{y}_{p,d} = \left[\beta_p L_p^{\text{récent}} + (1 - \beta_p) L_{p,d}^{\text{annuel}} \right] \times w_{p,j(d)} \times b_{p,d}, \quad (3.3)$$

où $w_{p,j(d)}$ est le poids du jour de semaine et $b_{p,d}$ le multiplicateur métier combiné. En régime normal, $\beta = 0,4$. Lorsqu'une hausse ou une baisse durable est détectée par comparaison de fenêtres récentes avec l'ancre annuelle, le poids récent est porté jusqu'à 0,85 ou 0,90. Le ratio de croissance annuel est borné entre 0,6 et 1,8 afin de contenir les valeurs extrêmes.

La fiabilité journalière est mesurée en rejouant les 28 derniers jours. Les paramètres sont réestimés chaque semaine et l'erreur porte sur les totaux hebdomadaires. Le produit est classé « Fiable », « Moyen », « Incertain », « Peu vendu » ou « Historique court ». Ces libellés évitent de présenter un pourcentage fragile pour une référence presque jamais vendue.

3.4.4 Combinaison des événements et commandes

Les multiplicateurs calendaires ne sont pas simplement multipliés les uns par les autres. Le système conserve le plus grand facteur supérieur à un et le plus petit facteur inférieur à un, puis les combine. Cette règle empêche qu'une fête, un match et un événement saisi le même jour produisent une hausse irréaliste.

Les commandes clients suivent une règle différente. Une commande future connue est une quantité certaine : elle est ajoutée à la prévision de la date et au MRP. En revanche, un pic historique détecté comme atypique est remplacé par une médiane locale pendant l'apprentissage. Cette séparation évite que le modèle interprète une commande exceptionnelle comme une nouvelle demande habituelle.

3.5 Conception du calcul MRP

3.5.1 Explosion des nomenclatures

Pour un produit p , une matière i et un mois t , le besoin brut est :

$$B_{i,t} = \sum_p Q_{p,t}^{\text{recommandée}} \times q_{p,i}, \quad (3.4)$$

où $q_{p,i}$ représente la quantité de matière i nécessaire pour une unité du produit. Lorsque la recette contient un produit semi-fini, celui-ci est remplacé récursivement par sa propre recette. Un ensemble d'alias unifie par exemple les différentes écritures d'une même farine. Les grammes et millilitres sont ensuite convertis en kilogrammes et litres pour le bon de commande.

L'algorithme applique l'ordre de priorité suivant : recette exacte validée, détection par le nom, recette générique de famille, puis absence explicite de recette. La provenance accompagne le résultat. Ainsi, une estimation reste utilisable pour le pilotage, mais elle n'est pas confondue avec une fiche validée par le chef.

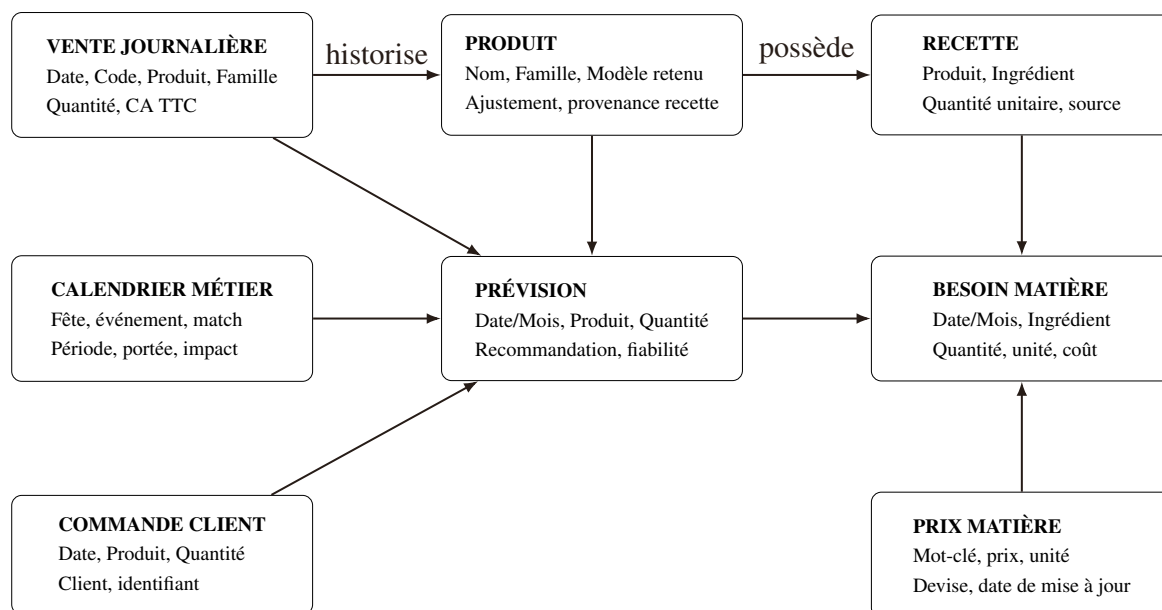
3.5.2 Valorisation

Les prix sont recherchés par mots-clés du plus spécifique au plus général. Le coût d'une ligne vaut la quantité normalisée multipliée par le prix unitaire. Le food-cost présenté dans le dashboard est un plancher, car il ne comprend ni main-d'œuvre, ni énergie, ni emballages absents du référentiel. Cette restriction est affichée à l'utilisateur au lieu d'être masquée.

3.6 Modèle logique des données

3.6.1 Relations principales

La solution n'impose pas une base applicative transactionnelle. Les tables logiques sont matérialisées par des CSV et JSON. La figure 3.4 montre les relations nécessaires au calcul. Les transformations tabulaires et temporelles s'appuient principalement sur pandas [4].

**FIGURE 3.4** – Modèle logique des données de PrevisionPaul

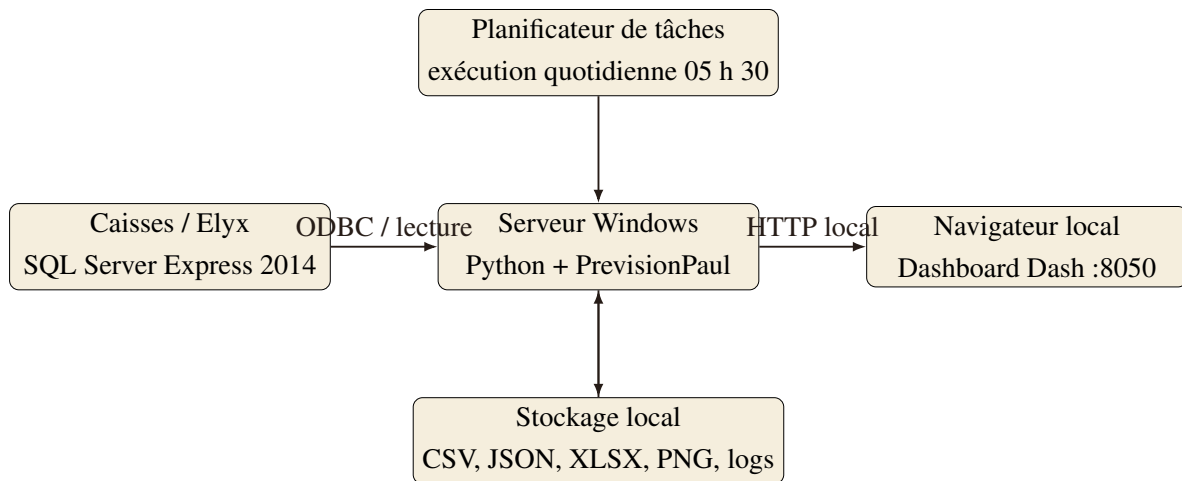
3.6.2 Dictionnaire synthétique

TABLE 3.3 – Dictionnaire des champs principaux

Champ	Type	Définition	Contrainte
Date	date ISO	Jour de vente ou de prévision.	Jour clôturé pour les ventes SQL.
Produit	texte	Libellé canonique de la référence.	Non vide ; rapproché des alias.
Famille	catégorie	Département commercial.	Une des familles connues.
Quantite	décimal	Quantité réellement vendue.	Valeur négative ramenée ou traitée comme retour.
Qty_Prev	décimal	Prévision centrale avant sécurité.	Supérieure ou égale à zéro.
Qty_Recommandee	décimal	Prévision augmentée selon l'incertitude.	Supérieure ou égale à Qty_Prev.
Fiabilite	catégorie	Niveau issu du backtest.	Fiable, Moyen, Incertain, Peu vendu ou Hist. court.
Ingredient	texte	Matière première canonique.	Alias résolus avant agrégation.
Quantite_Requise	décimal	Besoin total calculé.	Unité normalisée pour l'achat.
Modele	catégorie	Candidat retenu pour le produit.	Présent dans la liste des candidats.

3.7 Architecture matérielle et déploiement

La solution s'exécute sur un serveur ou un poste Windows relié au réseau interne. Le serveur SQL Elyx reste la source transactionnelle ; PrevisionPaul effectue une lecture sur les jours clôturés et conserve son propre historique analytique en fichiers. Le dashboard écoute par défaut sur le port local 8050. Le navigateur peut être ouvert sur la même machine, ce qui évite toute exposition Internet.

**FIGURE 3.5** – Diagramme de déploiement matériel

3.8 Conclusion

La conception retenue combine des scénarios automatisés, des moteurs spécialisés et des contrats de données simples. La validation temporelle, la réconciliation, la séparation des commandes exceptionnelles et l'explosion récursive des recettes répondent directement aux contraintes identifiées. Le chapitre suivant montre comment cette conception a été implémentée, testée et présentée aux utilisateurs.

Chapitre 4

Réalisation du système

Objectifs du chapitre

Ce chapitre présente l'environnement de développement, la réalisation de la chaîne de traitement, les interfaces principales, le déploiement, la stratégie de test et les résultats observés sur la version synchronisée du projet.

4.1 Introduction

La réalisation a été conduite par incréments. Nous avons d'abord sécurisé la lecture et la consolidation des ventes, puis construit les prévisions mensuelles et journalières. Les règles de fêtes, de commandes et de nomenclatures ont ensuite été intégrées avant la création du dashboard et de l'automatisation. Chaque étape a produit des fichiers contrôlables indépendamment de l'interface.

4.2 Environnement de développement

4.2.1 Environnement matériel

Le développement et la validation finale ont été réalisés sur la configuration du tableau 4.1. Le calcul reste principalement contraint par le nombre de produits et les backtests répétés ; il ne nécessite pas de carte graphique dédiée.

TABLE 4.1 – Configuration matérielle de développement et de validation

Composant	Configuration
Processeur	AMD Ryzen 5 7500F, 6 cœurs
Mémoire vive	15,6 Go
Stockage principal	SSD, capacité d'environ 930 Go
Architecture	64 bits
Système d'exploitation	Windows 11 Professionnel

Le serveur cible documenté est un environnement Windows relié à SQL Server Express 2014 et au progiciel Elyx Resto 11.15.5. La connexion utilise l'authentification Windows ; aucun mot de passe n'est inscrit dans le code.

4.2.2 Environnement logiciel

TABLE 4.2 – Environnement logiciel utilisé lors de la validation

Outil	Version	Usage
Python	3.11.9	Langage principal et orchestration.
pandas	2.2.2	Lecture, nettoyage, agrégation et transformation tabulaire.
NumPy	1.26.4	Calcul numérique et manipulation des vecteurs de prévision.
statsmodels	0.14.6	Holt–Winters, Theta et ARIMA.
scikit-learn	1.8.0	Modèle global de gradient boosting.
Dash	4.3.0	Application web et callbacks interactifs.
Plotly	6.7.0	Graphiques interactifs.
openpyxl	3.1.5	Lecture et génération de classeurs Excel.
PyODBC	dépendance requis	Connexion au SQL Server Elyx.
Pytest	dépendance de test	Exécution de la suite automatisée.
Git / GitHub	dépôt distant	Versionnement et déploiement du code.

4.3 Réalisation de la chaîne de données

4.3.1 Import historique

Le script de conversion reconstruit le fichier `ventes_journalieres.csv` à partir des exports bruts. Il traite l'encodage hérité, associe les familles, retire les sous-totaux et agrège les lignes par date et produit. Les colonnes finales sont volontairement stables : Date, Code, Produit, Famille, Quantité et CA TTC. Cette stabilité permet aux moteurs et au dashboard de partager la même source de vérité journalière.

4.3.2 Export SQL défensif

Le module `exporter_ventes_sql.py` recherche les tables `IMPUTATION_<site>`, sélectionne les journées clôturées et agrège les lignes de ticket. Les ventes SQL remplacent les lignes

de même date dans le CSV, mais laissent les autres dates intactes. Avant toute écriture, le module vérifie la présence des colonnes attendues, le volume retourné et la cohérence des totaux. Si les caisses ne sont pas encore raccordées ou si la requête ne renvoie rien, l'opération devient un no-op : le fichier historique n'est pas écrasé.

4.3.3 Configuration externalisée

Les éléments métier sont stockés dans `data/*.json`. Cette décision évite de mélanger les paramètres et le code. La version auditée contient notamment 40 périodes de fêtes marocaines, 848 choix de modèle, 270 entrées de recettes dans le référentiel principal et 86 groupes d'alias matières. Le dashboard reconnaît 261 recettes comme exactes et correctement rattachées parmi 1 109 produits prévus ; les autres sont signalées comme estimées ou manquantes.

4.4 Réalisation des prévisions

4.4.1 Pipeline mensuel

Le point d'entrée `main.py` appelle `pipeline.run()`. Le pipeline crée un dossier d'export daté, charge et nettoie les données, calcule les prévisions agrégées, boucle sur les produits, applique le modèle global si nécessaire, réconcilie les niveaux, ajoute les ajustements métier, estime l'incertitude et calcule les besoins matières.

Le pseudo-code suivant résume l'enchaînement sans exposer les détails de chaque modèle :

Listing 4.1 – Pseudo-code du pipeline mensuel

```
1  donnees = charger_et_nettoyer_sources()
2  historique = agreger_par_mois_et_par_produit(donnees)
3
4  for produit in produits:
5      modele = modele_selectionne(produit)
6      prevision[produit] = prevoir(modele, historique[produit])
7
8  prevision = appliquer_modele_global_si_retenu(prevision)
9  prevision = reconcilier_sur_agregat(prevision)
10 prevision = appliquer_fetes_evenements_commandes(prevision)
11 plan = ajouter_intervalles_et_marge_de_securite(prevision)
12 besoins = explorer_nomenclatures(plan)
13 exporter(plan, besoins, validations, graphiques)
```

4.4.2 Prévision journalière et suivi

Le module journalier génère un enregistrement pour chaque date et produit sur 62 jours. Il conserve la prévision centrale, la quantité recommandée, la fiabilité et la part provenant d'une commande client. Le module `suivi.py` reconstruit ensuite ce que le système aurait prévu au cours des 28 derniers jours en n'utilisant, à chaque origine, que les observations disponibles auparavant. Cette méthode empêche toute fuite d'information future.

4.4.3 Gestion de l'incertitude

La recommandation ne se limite pas à la prévision moyenne. Pour un niveau de service cible de 95 %, un quantile de la loi normale transforme l'écart-type de l'erreur en marge. Lorsque l'historique est insuffisant, une marge prudente par défaut est appliquée et l'étiquette de fiabilité reste explicite. La quantité recommandée répond ainsi à un objectif anti-rupture, tandis que la prévision centrale reste disponible pour l'analyse.

4.5 Réalisation du calcul matières

4.5.1 Recettes et produits semi-finis

Les recettes sont représentées par des dictionnaires produit–ingrédient. Une recette peut référencer un produit semi-fini, par exemple une pâte, une crème ou une sauce. La fonction d'explosion parcourt ces références récursivement, protège contre les cycles et additionne les matières identiques après résolution des alias. Les derniers changements du dépôt renforcent cette logique pour les préparations maison de cuisine et les libellés contenant un code ou une marque interne.

Le workflow de validation par le chef comprend deux scripts. Le premier génère un classeur par catégorie avec la recette disponible et sa provenance ; le second importe uniquement les lignes explicitement validées, sauvegarde l'ancien JSON et met à jour la provenance. Cette séparation réduit le risque qu'une estimation soit promue automatiquement en donnée exacte.

4.5.2 Exports opérationnels

Le pipeline produit notamment :

- `plan_production_securise.csv`, avec prévision, fourchette, recommandation, fiabilité et modèle ;
- `besoins_ingredients_planifies.csv`, avec date, ingrédient et quantité requise ;

- `previsions_PAUL_2026.xlsx`, classeur de synthèse ;
- `bon_de_commande_gérant.txt`, support directement lisible ;
- les validations mensuelles de quantité et de chiffre d'affaires ;
- des graphiques statiques pour le responsable et le contrôle terrain.

4.6 Principales interfaces graphiques

4.6.1 Accueil et synthèse

La page d'accueil, présentée à la figure 4.1, fournit une lecture immédiate : quantité à produire, chiffre d'affaires prévu, prochaine fête, part des prévisions fiables et écart prévu-réel. Elle signale également la fraîcheur des ventes. Cette alerte est essentielle dans la capture : les données sont arrêtées au 28 juin 2026, soit quarante jours avant la date d'affichage.

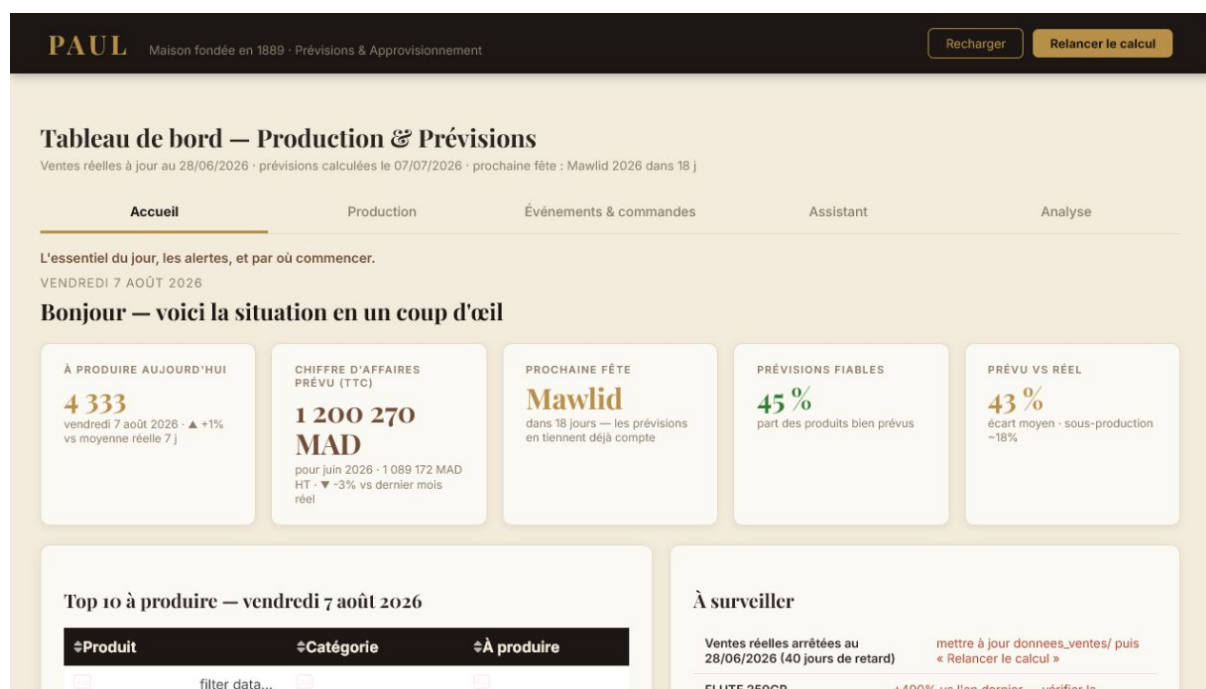


FIGURE 4.1 – Page d'accueil du tableau de bord PrevisionPaul

4.6.2 Production journalière

La vue de production du jour affiche les quantités recommandées et permet de choisir une date, une catégorie et un export Excel. Trois indicateurs résument le volume du jour, le nombre de références à préparer et l'horizon disponible. La prévision centrale et la fiabilité restent accessibles dans le tableau afin que l'utilisateur distingue le besoin moyen de la marge de sécurité.

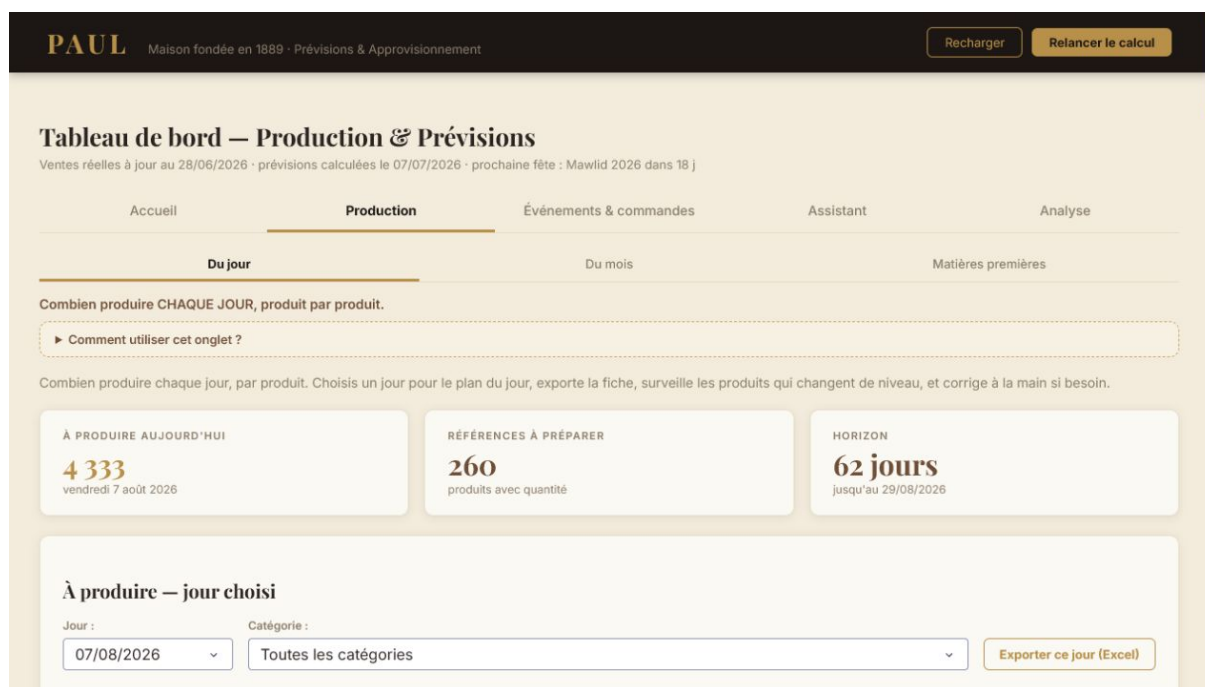


FIGURE 4.2 – Vue de production journalière

4.6.3 Matières premières

La vue MRP présente 236 ingrédients distincts pour le mois calculé. Elle regroupe les solides, les liquides, le budget d'achat estimé, le food-cost plancher et la couverture des prix. Le tableau détaillé permet de contrôler les matières les plus importantes. Une seconde section classe les recettes à saisir en priorité selon le poids matière qu'elles représentent.

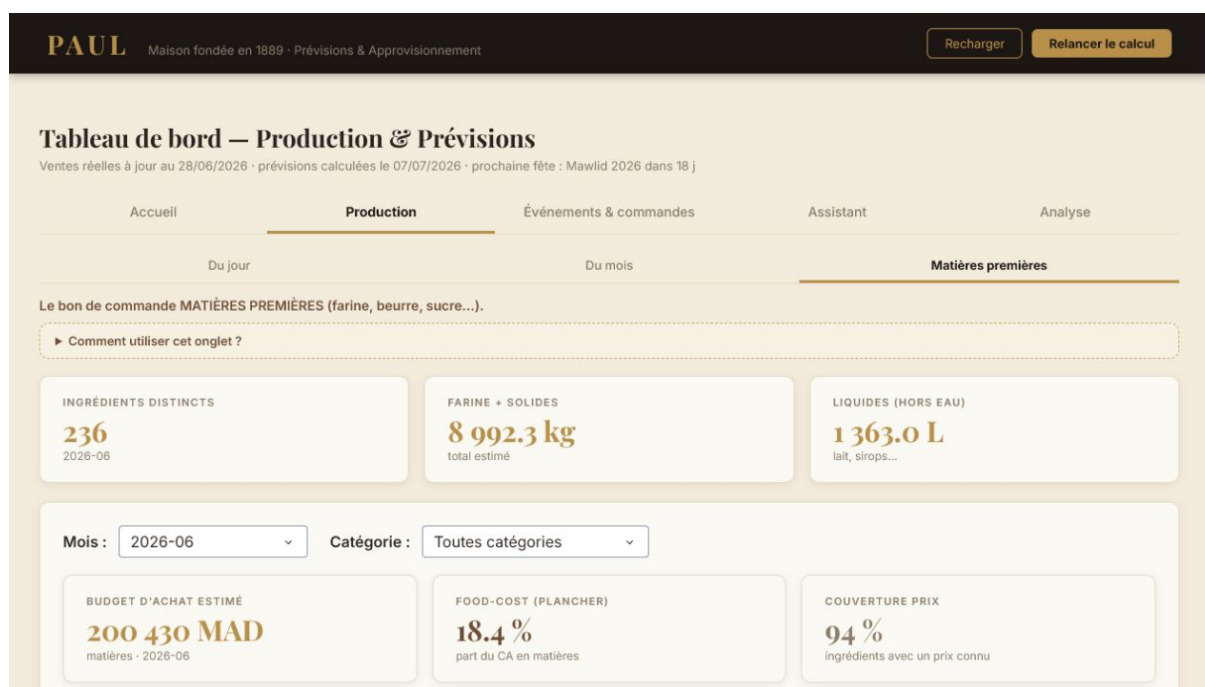


FIGURE 4.3 – Vue des besoins en matières premières

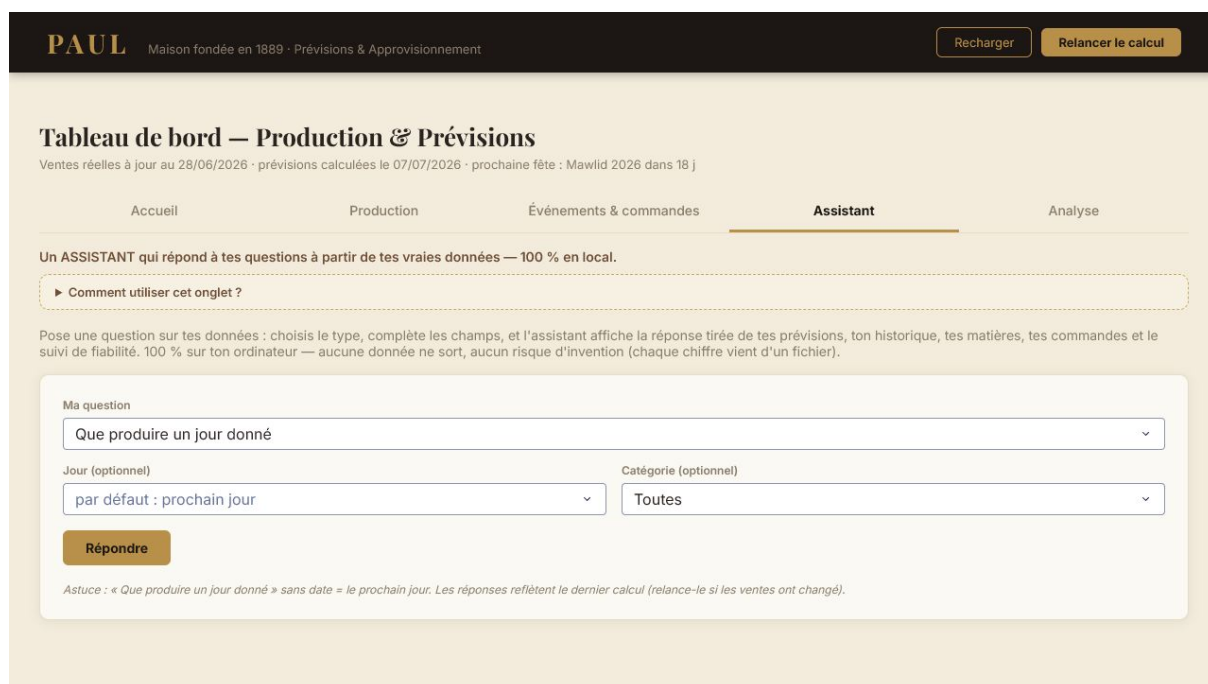
4.6.4 Événements et commandes

L'utilisateur peut déclarer un match, un jour férié, un concert ou un autre événement. Pour un match, l'impact est appris à partir des occurrences passées et pondéré par l'importance de la rencontre ; pour les autres types, un impact manuel peut être saisi. L'onglet voisin enregistre les commandes clients datées.

FIGURE 4.4 – Gestion des événements et des matchs

4.6.5 Assistant déterministe local

La figure 4.5 montre l'assistant guidé. L'utilisateur choisit un type de question et renseigne uniquement les champs nécessaires. La fonction correspondante lit les fichiers réels et renvoie un texte et, si besoin, un tableau. Ce choix privilégie la confidentialité et l'absence d'hallucination par rapport à une conversation libre avec un service externe.



PAUL Maison fondée en 1889 · Prévisions & Approvisionnement

Recharger Relancer le calcul

Tableau de bord — Production & Prévisions

Ventes réelles à jour au 28/06/2026 · prévisions calculées le 07/07/2026 · prochaine fête : Mawlid 2026 dans 18 j

Accueil Production Événements & commandes **Assistant** Analyse

Un ASSISTANT qui répond à tes questions à partir de tes vraies données — 100 % en local.

► Comment utiliser cet onglet ?

Pose une question sur tes données : choisis le type, complète les champs, et l'assistant affiche la réponse tirée de tes prévisions, ton historique, tes matières, tes commandes et le suivi de fiabilité, 100 % sur ton ordinateur — aucune donnée ne sort, aucun risque d'invention (chaque chiffre vient d'un fichier).

Ma question

Que produire un jour donné

Jour (optionnel) Catégorie (optionnel)

par défaut : prochain jour Toutes

Répondre

Astuce : « Que produire un jour donné » sans date = le prochain jour. Les réponses reflètent le dernier calcul (relance-le si les ventes ont changé).

FIGURE 4.5 – Assistant local fondé sur les données calculées

4.6.6 Analyse du chiffre d'affaires et des marges

L'espace Analyse rassemble l'historique, les prévisions de chiffre d'affaires et les marges matière par produit. Les cartes indiquent notamment un chiffre d'affaires historique HT de 51,53 millions MAD sur les données consolidées par le dashboard, un mois moyen de 873 399 MAD et les prévisions 2026. Les marges restent qualifiées d'indicatives tant que toutes les recettes et tous les prix ne sont pas validés.

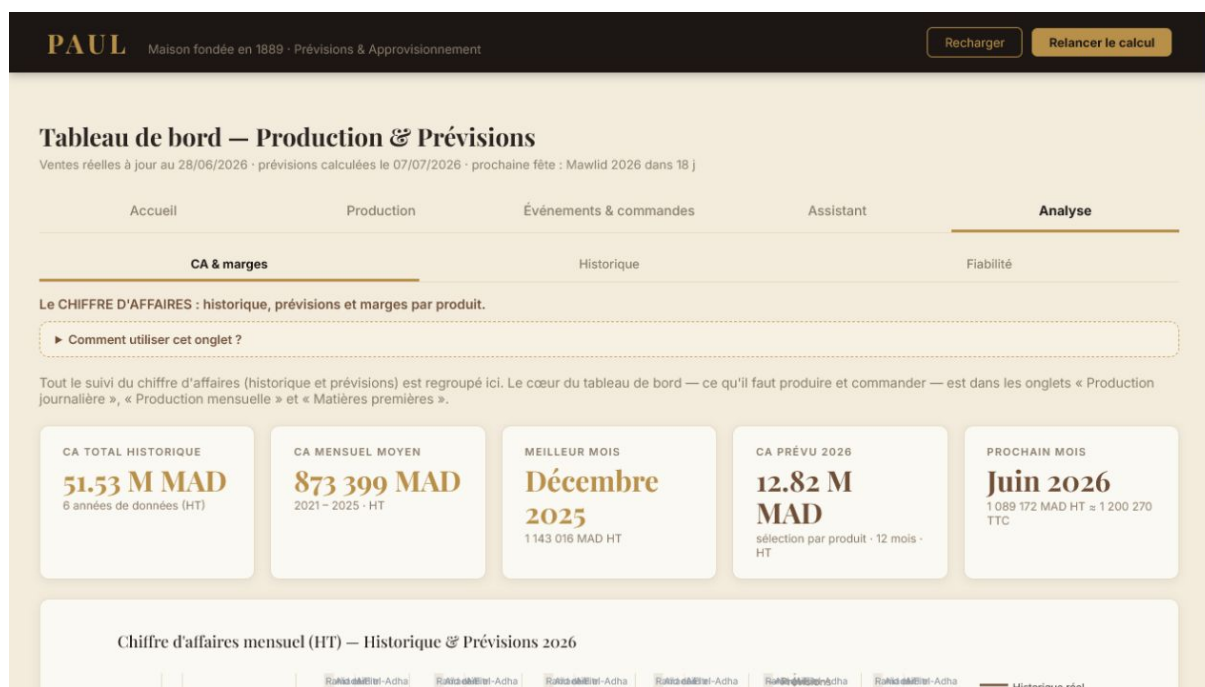


FIGURE 4.6 – Analyse du chiffre d'affaires et des marges matière

4.7 Automatisation et déploiement

Le dépôt fournit un lanceur double-clic pour l'utilisateur final et un script d'installation de raccourci avec l'icône PAUL. En production, le code est prévu dans `C:\PrevisionPaul`, hors des dossiers personnels. Une tâche Windows nommée « PrevisionPaul - MAJ quotidienne » exécute le script `mise_a_jour_quotidienne.py` à 05 h 30. Les étapes et erreurs sont écrites dans un fichier `logs/auto_AAAAMMJJ.log`.

Le code est synchronisé par Git, tandis que les ventes, recettes et prix sensibles restent exclus du dépôt public. Cette séparation simplifie les mises à jour : un `git pull` actualise l'application sans déplacer les données locales.

4.8 Tests et validation

4.8.1 Stratégie de test

La suite Pytest couvre les règles les plus sensibles : préparation des données, prévisions, fêtes, commandes, incertitude, nomenclatures, coûts, couverture, assistant et exports. Un test spécifique vérifie que l'assistant ne dépend d'aucun service réseau. D'autres tests contrôlent l'ajout exact d'une commande, la neutralisation des pics, les unités de matière et l'absence de régression dans les tables générées.

La commande `python -m pytest tests -q` a été exécutée après la synchronisation GitHub. Les 86 tests ont réussi en 7,85 secondes sur la machine de validation.

Résultat de la recette technique**86 tests réussis sur 86**

Aucun échec détecté sur le commit be291f7.

4.8.2 Validation mensuelle

La validation walk-forward disponible rejoue douze mois, de juin 2025 à mai 2026. Elle confirme que Holt–Winters est le meilleur modèle agrégé parmi les cinq séries de référence du fichier de validation : WAPE de 8,25 % sur les quantités et 8,28 % sur le chiffre d'affaires. Cette mesure ne remplace pas la sélection par produit ; elle contrôle la cohérence du niveau global.

TABLE 4.3 – Résultats de la validation mensuelle agrégée

Modèle	WAPE quantité	WAPE CA
Moyenne mobile	9,36 %	11,09 %
Tendance	15,18 %	19,03 %
Naïf saisonnier	19,31 %	22,92 %
Décomposition saisonnière	16,22 %	20,49 %
Holt–Winters	8,25 %	8,28 %



FIGURE 4.7 – Graphique de validation mensuelle généré par le pipeline

4.8.3 Suivi journalier

Le suivi du 1^{er} au 28 juin 2026 contient 30 905 comparaisons produit-date. À l’échelle de toutes les lignes, le WAPE atteint 43,47 %, ce qui reflète la difficulté de la longue traîne et des petits volumes. Après agrégation par jour, le WAPE est de 18,62 % et le biais de −18,12 %, indiquant une sous-prévision globale sur cette fenêtre. Ce diagnostic est affiché plutôt que masqué : il justifie la marge de sécurité et la nécessité de mettre à jour les ventes, arrêtées au 28 juin dans la capture du 7 août.

TABLE 4.4 – Indicateurs observés dans les exports du 7 juillet 2026

Indicateur	Valeur
Produits dans le plan mensuel	1 164
Prévision mensuelle centrale totale	95 898 unités
Quantité mensuelle recommandée totale	128 239 unités
Part des lignes mensuelles classées fiables	44,8 %
Horizon de prévision journalière	62 jours
Produits dans l'export journalier	1 109
Ingrédients distincts calculés	236
Budget matière estimé affiché	200 430 MAD
Couverture des prix affichée	94 %
Tests automatisés	86 / 86 réussis

Les résultats chiffrés correspondent aux fichiers disponibles au 7 juillet 2026. Ils ne constituent pas une prévision actualisée au 7 août 2026, car les ventes réelles s'arrêtent au 28 juin. Le dashboard détecte et affiche ce retard. Une mise à jour SQL ou fichier, suivie d'un recalcul, est nécessaire avant toute décision opérationnelle réelle.

4.9 Difficultés rencontrées et solutions

4.9.1 Hétérogénéité et qualité des données

Les exports historiques combinent plusieurs années, sites, encodages et formats. Les sous-totaux pouvaient doubler les volumes. Nous avons stabilisé un schéma CSV, introduit des contrôles de seuil, consolidé les mois récents et conservé un référentiel de nettoyage. Le système refuse désormais de remplacer un historique valide par une extraction vide.

4.9.2 Longue traîne et changements de régime

Plusieurs centaines de produits ont un historique court ou intermittent. Les modèles classiques deviennent instables et une erreur relative sur un volume presque nul perd son sens. Nous avons combiné Croston, des replis simples, un GBM global et des étiquettes spécifiques. Pour les références qui changent rapidement de niveau, le modèle journalier augmente le poids de la fenêtre récente tout en bornant la réaction à un pic isolé.

4.9.3 Recettes incomplètes et unités

La qualité du MRP dépend directement des nomenclatures. Des variations de casse, des codes internes et des unités incohérentes empêchaient certains rapprochements. Nous avons ajouté des alias, une normalisation des unités, une gestion récursive des PSF et un workflow de validation par le chef. Les derniers correctifs du dépôt traitent notamment une erreur d'échelle kilogramme/gramme et plusieurs familles de préparations cuisine.

4.9.4 Lisibilité de l'interface

Le dashboard regroupe de nombreux usages dans un seul fichier d'application. Une navigation plate aurait créé trop d'onglets. Nous avons donc organisé les vues en cinq groupes, ajouté des sous-onglets thématiques, des aides contextuelles et des cartes de navigation. Les lectures coûteuses sont mises en cache puis invalidées après un recalcul.

4.10 Conclusion

La réalisation concrétise l'architecture décrite au chapitre précédent : sources contrôlées, moteurs séparés, règles métier explicites, exports traçables et interface orientée décision. Les résultats mensuels sont satisfaisants au niveau agrégé et la suite automatisée valide les comportements critiques. Le suivi journalier met toutefois en évidence un biais récent et rappelle qu'un outil de prévision doit être surveillé et recalibré en continu.

Conclusion générale

L'objectif de ce projet était de transformer un historique de ventes hétérogène en un outil local capable d'aider concrètement les équipes de PAUL Maroc à décider quoi produire et quoi commander. Pour atteindre cet objectif, nous avons construit une chaîne complète qui consolide les ventes journalières et mensuelles, compare dix candidats de prévision selon le produit, réconcilie les résultats avec un niveau global, intègre les fêtes marocaines, les matchs, les événements et les commandes clients, puis convertit les quantités recommandées en besoins de matières premières grâce aux recettes et aux produits semi-finis. Le résultat prend la forme de plans quotidien et mensuel sécurisés, d'un bon de commande, d'indicateurs de coût et d'une interface accessible. La version étudiée traite 373 875 lignes de ventes, couvre plus de mille produits, sélectionne un modèle pour 848 références et calcule 236 ingrédients distincts. La validation mensuelle glissante obtient un WAPE de 8,25 % sur les quantités agrégées avec Holt–Winters, tandis que les 86 tests automatisés réussissent sur le dernier commit synchronisé. Ces résultats montrent que l'architecture répond aux besoins fondamentaux de reproductibilité, de traçabilité et de confidentialité. Ils doivent cependant être interprétés avec esprit critique. La précision journalière se dégrade sur la longue traîne et la fenêtre récente révèle une sous-prévision globale ; les ventes disponibles lors de l'audit s'arrêtent au 28 juin 2026 ; une partie des recettes et des prix reste estimée ; enfin, le food-cost calculé ne comprend pas encore toutes les charges. Le dashboard rend ces limites visibles afin que l'utilisateur ne confonde pas une recommandation avec une certitude. Les prochaines améliorations devraient donc porter en priorité sur l'alimentation SQL continue, la validation systématique des recettes par le chef, l'intégration des stocks réels et des délais fournisseurs, le suivi de la casse et des invendus, et la mise en place d'alertes lorsque le biais dépasse un seuil. Une évolution vers un stockage relationnel des configurations et des historiques d'exécution faciliterait également le multi-site et la gestion des droits. Enfin, un modèle local de langage pourrait enrichir l'assistant en langage naturel, à condition de conserver les outils déterministes comme seule source de chiffres et de ne jamais faire sortir les données de l'entreprise. Ce projet nous a permis d'articuler ingénierie logicielle, données, séries temporelles et contraintes opérationnelles.

Bibliographie et nétographie

- [1] HYNDMAN, Rob J. et ATHANASOPOULOS, George. *Forecasting : Principles and Practice*, 3^e édition, Melbourne, OTexts, 2021. Version en ligne consultée le 7 août 2026 : <https://otexts.com/fpp3/>.
- [2] OBJECT MANAGEMENT GROUP. *OMG Unified Modeling Language, Version 2.5.1*, décembre 2017. Spécification consultée le 7 août 2026 : <https://www.omg.org/spec/UML/2.5.1>.
- [3] ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR. *Template du rapport de projet de fin d'année — 4e année IIR*, document pédagogique interne, année universitaire 2025–2026.

Nétographie et documentation technique

- [4] PANDAS DEVELOPMENT TEAM. *pandas User Guide*. Documentation officielle consultée le 7 août 2026 : https://pandas.pydata.org/docs/user_guide/.
- [5] STATSMODELS DEVELOPERS. *statsmodels.tsa.holtwinters.ExponentialSmoothing*. Documentation officielle consultée le 7 août 2026 : <https://www.statsmodels.org/dev/generated/statsmodels.tsa.holtwinters.ExponentialSmoothing>.
- [6] SCIKIT-LEARN DEVELOPERS. *GradientBoostingRegressor*. Documentation officielle consultée le 7 août 2026 : <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.GradientBoostingRegressor.html>.
- [7] PLOTLY. *Dash for Python Documentation — Layout*. Documentation officielle consultée le 7 août 2026 : <https://dash.plotly.com/layout>.
- [8] MOUSSAWI, Othmane. *PrevisionPaul — dépôt du projet*. Version be291f7, consultée le 7 août 2026 : <https://github.com/OthmaneW37/PrevisionPaul>.

Annexe A

Guide d'installation et d'exploitation

A.1 Installation locale

Le code et les données doivent être séparés. Le code peut être cloné depuis le dépôt, tandis que les dossiers sensibles sont transférés directement sur la machine de l'entreprise.

Listing A.1 – Commandes d'installation

```
1 git clone https://github.com/0thmaneW37/PrevisionPaul.git
2 cd PrevisionPaul
3 python -m pip install -r requirements.txt
4 python -m pytest tests -q
```

Les dossiers `data/` et `donnees_ventes/` doivent ensuite être placés dans la racine du projet. Ils ne sont pas publiés avec le code lorsqu'ils contiennent des informations métier.

A.2 Commandes d'exploitation

TABLE A.1 – Principales commandes d'exploitation

Commande	Effet
<code>pythondashboard.py</code>	Lance le tableau de bord sur <code>http://127.0.0.1:8050</code> .
<code>pythonmain.py</code>	Recalcule les prévisions mensuelles, le plan sécurisé et le MRP.
<code>python-mpaul_forecast.forecast_journalier</code>	Régénère l'horizon journalier de 62 jours.
<code>pythonutils/exporter_ventes_sql.py</code>	Lit les jours clôturés depuis SQL Server et fusionne le CSV.
<code>pythonutils/mise_a_jour_quotidienne.py</code>	Exécute export SQL, journalier puis mensuel avec journalisation.
<code>pythonutils/generer_tableau_recettes.py</code>	Crée le classeur à compléter par le chef.
<code>pythonutils/importer_recettes_chefs.py</code>	Importe les recettes explicitement validées.
<code>python-mpytesttests-q</code>	Lance l'ensemble des tests automatisés.

A.3 Procédure de mise à jour quotidienne

1. vérifier que la base Elyx est accessible ou déposer le nouvel export dans `donnees_ventes/`;
2. lancer la chaîne quotidienne ou attendre la tâche de 05 h 30;
3. consulter le journal du jour dans `logs/`;
4. ouvrir le dashboard et contrôler la date « ventes réelles à jour au »;
5. examiner les alertes de niveau et la fiabilité avant d'exporter le plan;
6. vérifier le bon de commande avec le chef lorsque des recettes sont encore estimées.

Annexe B

Structure du projet et fichiers produits

B.1 Arborescence fonctionnelle

```

PrevisionPaul/
|-- dashboard.py           # interface Dash
|-- main.py               # point d'entree mensuel
|-- watcher.py           # surveillance des fichiers
|-- paul_forecast/        # moteurs et regles metier
|-- data/                 # configuration JSON locale
|-- donnees_ventes/       # historique des ventes
|-- exports/              # sorties journalieres et datees
|-- outils/               # imports, benchmarks, recettes
|-- tests/                # suite Pytest
|-- docs/                 # documents pour les equipes
'-- Rapport/              # sources LaTeX du present rapport

```

B.2 Exports principaux

TABLE B.1 – Fichiers produits par le système

Fichier	Contenu
exports/previsions_journalieres.csv	Date, produit, famille, prévision, recommandation, fiabilité et commande.
exports/suivi_prevu_reel.csv	Comparaison hors échantillon sur les jours récents.
plan_production_securise.csv	Plan mensuel par produit, modèle, erreur et intervalle.
besoins_ingredients_planifies.csv	Quantité requise par ingrédient et par mois.
previsions_detaillees_par_produit.csv	Historique et scénarios de prévision détaillés.
previsions_par_famille.csv	Synthèse mensuelle par famille.
validation_mensuelle_quantite.csv	Réel et prévisions walk-forward en quantité.
validation_mensuelle_ca.csv	Réel et prévisions walk-forward en chiffre d'affaires.
previsions_PAUL_2026.xlsx	Classeur multi-feuilles à destination du responsable.
bon_de_commande_gerant.txt	Synthèse textuelle des matières à commander.

Annexe C

Compléments de validation

C.1 Répartition des modèles sélectionnés

Le benchmark disponible contient 848 produits. La répartition du meilleur modèle montre l’intérêt d’une sélection hétérogène : Theta est retenu pour 293 produits, le GBM pour 227, le naïf saisonnier pour 102, ETS pour 77, la moyenne mobile pour 44, l’ensemble médian pour 35, la moyenne pour 23, ARIMA pour 20, Croston pour 14 et l’ensemble moyen pour 13.

TABLE C.1 – Répartition des modèles retenus dans le benchmark

Modèle	Produits	Part	Modèle	Produits	Part
Theta	293	34,6 %	Moyenne mobile	44	5,2 %
GBM	227	26,8 %	Ensemble médian	35	4,1 %
Naïf saisonnier	102	12,0 %	Moyenne	23	2,7 %
ETS	77	9,1 %	ARIMA	20	2,4 %
Croston	14	1,7 %	Ensemble moyen	13	1,5 %

C.2 Graphiques statiques générés

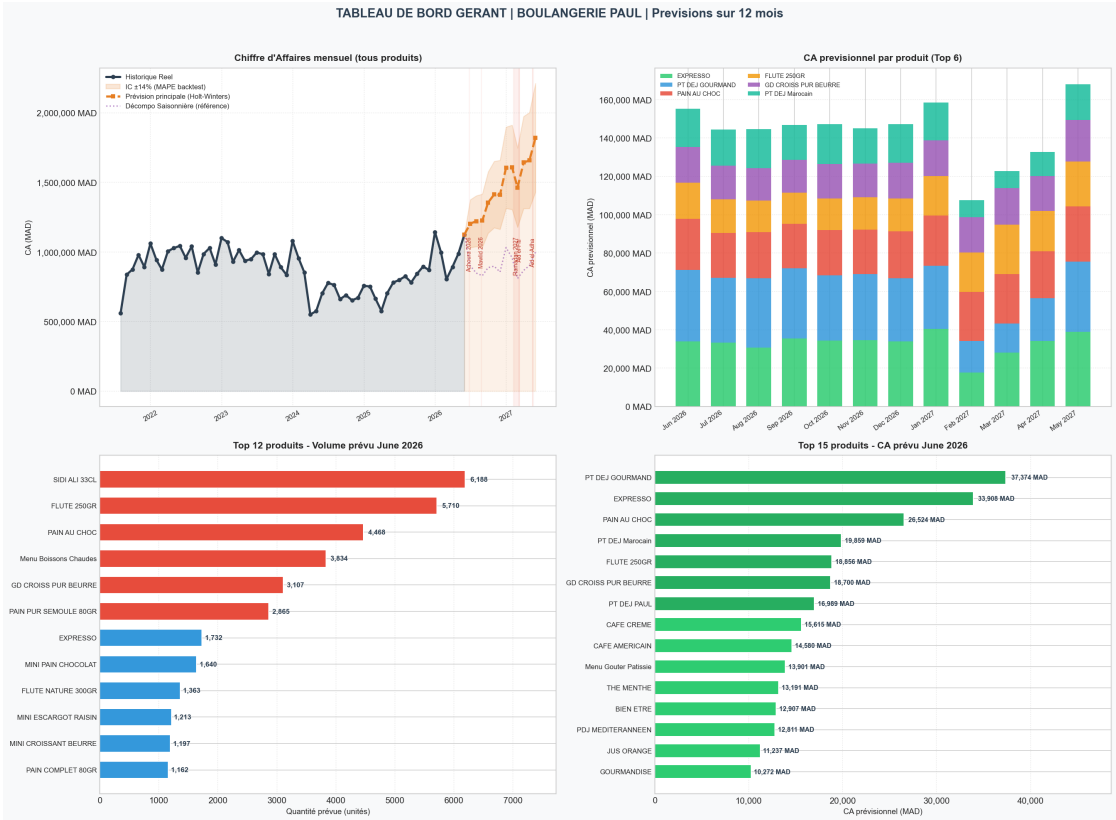


FIGURE C.1 – Dashboard statique destiné au responsable

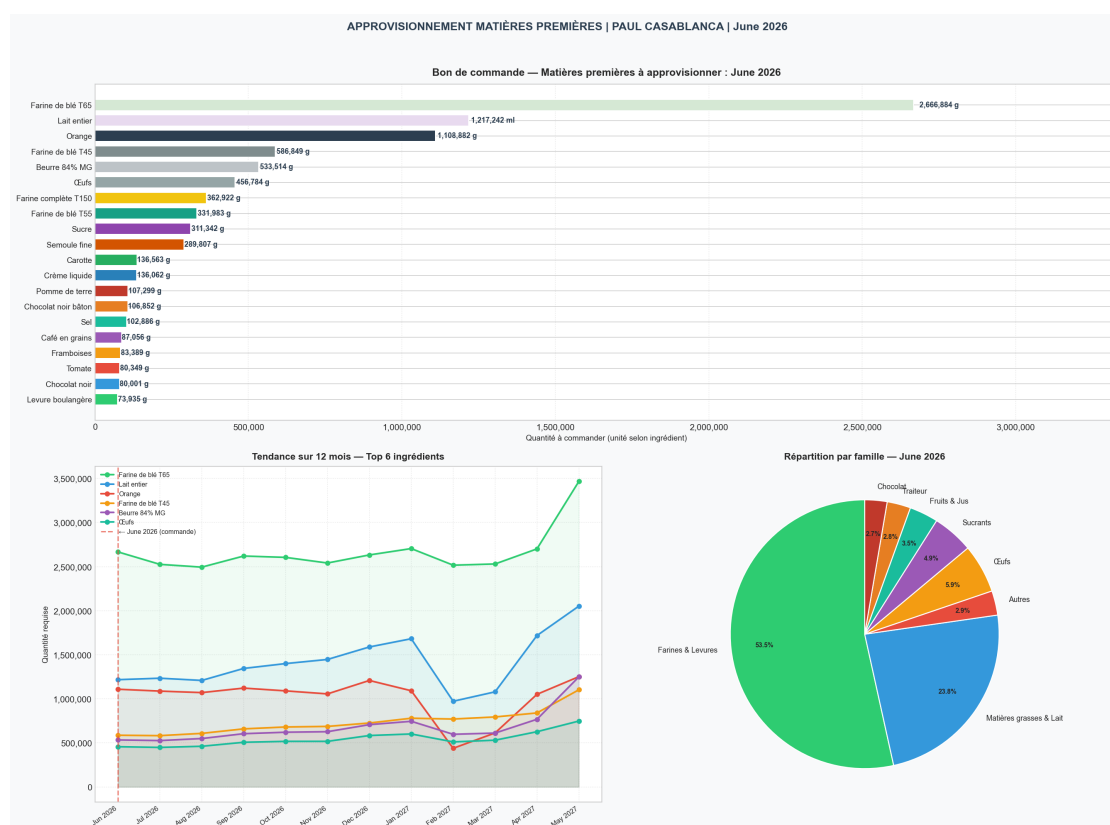


FIGURE C.2 – Dashboard statique des matières premières

C.3 Limites à contrôler avant usage opérationnel

- actualiser les ventes et vérifier la date de fraîcheur ;
- confirmer les recettes à plus fort poids matière ;
- remplacer les prix estimés par les factures fournisseurs ;
- retirer le calibrage provisoire des farines après validation complète ;
- analyser le biais journalier et recalibrer les seuils si nécessaire ;
- ne pas interpréter les marges matière comme une marge nette ;
- contrôler les événements et commandes futurs avant chaque recalcul.